

Top-K Memory Retrieval: Scaling, Worked Out by Hand

Why brute-force recall collapses as long-term memory grows — counting the operations behind `store.asearch(ns, query, limit=5)`.

What this document does. It takes the one line of recall code from Module 2.11 — `hits = await store.asearch(ns, query=query, limit=5)`, the top- K semantic read over a user’s long-term memory — and counts what it actually costs as the store grows. We fix a tiny model (N stored vectors, dimension D , top- K), derive the brute-force operation count $O(N \cdot D)$, derive why selecting the top- K wants a heap ($O(N \log K)$) not a full sort ($O(N \log N)$), and then derive the approximate-nearest-neighbour (ANN / HNSW) cost $O(\log N \cdot D)$. We tabulate the millions-to-billions blow-up of brute force against the near-constant ANN at $N = 10^3, 10^6, 10^9$ and $D = 768$. Every number is reproduced by the program in Appendix A.

Where this sits. This is **Topic 2 of Module 2.11** (Memory, Guardrails, Safety) — the performance story behind the long-term memory layer and the anti-pattern “no TTL — memory grows unboundedly forever.” Its companions: Module 2.11 Topic 1 (namespace collisions — the *safety* of the same store) and the embeddings/RAG modules where the $O(N \cdot D)$ similarity loop first appears.

Concepts covered, in order: the recall as N similarities $\cdot$ cost per similarity $O(D)$ $\cdot$ brute force $O(N \cdot D)$ $\cdot$ full sort $O(N \log N)$ vs. a size- K heap $O(N \log K)$ $\cdot$ ANN candidate comparisons $O(\log N)$ $\cdot$ the asymptotic gap $\cdot$ why an unbounded memory store makes brute force fail.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one recall over a growing store

When the agent wants to “recall” relevant memories, it embeds the query into a D -dimensional vector and asks the store for the K stored memories most similar to it. In Module 2.11 that is `store.asearch(ns, query, limit=5)` — so $K = 5$. The store holds N memory vectors for this (`tenant`, `user`) namespace, and *N only grows*: every durable fact the agent learns is another vector, and (per the module’s anti-pattern) without a TTL it never shrinks. The cost of one recall is therefore a moving target. We count it.

Quantity	Symbol	Value here
Stored memory vectors in the namespace	N	$10^3, 10^6, 10^9$
Embedding dimension	D	768
Memories returned (top-K)	K	5
Cost of one similarity (dot product)	—	$O(D)$
Brute-force recall cost	—	$O(N \cdot D)$
ANN recall cost (HNSW)	—	$O(\log N \cdot D)$

Notation. We count *multiply-add operations* for the distance work, the part that dominates. A cosine or dot-product similarity between two D -vectors is D multiply-adds, so we write its cost

as exactly D . “Ops” below means these multiply-adds; constant factors and the log base are absorbed into the $O(\cdot)$ but we keep base-2 logs for concrete numbers.

Intuition

Brute force asks every memory “how similar are you?” — so its work scales with *everything the agent has ever stored*. ANN instead keeps a graph (HNSW) so the query only “walks” toward its neighbours, touching $O(\log N)$ vectors instead of all N . The difference is the difference between reading every page of a library to find one book, and following the catalogue.

2 Step 1 — brute force: N similarities, each $O(D)$

The exact (brute-force) top- K does two things. First, it computes the similarity between the query and *every* stored vector. There are N vectors and each similarity is a dot product over D dimensions — D multiply-adds. So the similarity phase is

$$\text{ops}_{\text{sim}}(N, D) = N \cdot D, \quad (1)$$

which is $O(N \cdot D)$ and is the dominant term. Second, it *selects* the top- K from those N scores. That selection is where a naive implementation wastes work, and Step 2 fixes it.

Mini-example — this operation in isolation

With $N = 1000$ and $D = 768$, the similarity phase is $1000 \times 768 = 768,000$ multiply-adds for a *single* recall. That is already three-quarters of a million operations to answer one “what do I remember?” — and it is the *small* store. Every memory the agent adds makes the next recall linearly more expensive.

3 Step 2 — selecting top-K: full sort vs. a heap

Once we have N scores, how do we pick the best K ? The obvious way is to *sort* all N scores and take the first K . A comparison sort costs

$$\text{ops}_{\text{sort}}(N) = N \log_2 N = O(N \log N). \quad (2)$$

But we do not need the whole order — only the top K . Keep a min-heap of size K : scan the N scores once, and for each, compare against the heap’s smallest and (if larger) replace it. Each of the N items does $O(\log K)$ heap work, giving

$$\boxed{\text{ops}_{\text{select}}(N, K) = N \log_2 K = O(N \log K).} \quad (3)$$

Since $K = 5$ is tiny and *fixed* while N explodes, $\log_2 K$ is a small constant ($\log_2 5 \approx 2.32$) — the heap replaces the growing $\log_2 N$ factor with a constant.

Mini-example — this operation in isolation

At $N = 10^6$: full sort costs $N \log_2 N = 10^6 \times 19.93 \approx 19,931,569$ comparisons; the size-5 heap costs $N \log_2 5 = 10^6 \times 2.32 \approx 2,321,928$ — about $8.6\times$ fewer. ✓ The heap is the right

default whenever $K \ll N$, which for memory recall it always is. Note that selection ($\sim 10^7$) is still dwarfed by the similarity phase ($N \cdot D = 7.68 \times 10^8$); distance work is the real cost.

4 Step 3 — ANN: $O(\log N)$ candidate comparisons

A graph index such as HNSW does not score all N vectors. It stores the vectors as nodes in a navigable small-world graph and, at query time, greedily hops from node to neighbouring node toward the query. The number of hops — and therefore the number of vectors actually compared — scales like the graph’s diameter, which for these constructions is $O(\log N)$. Each comparison is still a D -dimensional distance, so

$$\text{ops}_{\text{ANN}}(N, D) = O(\log_2 N) \cdot D. \quad (4)$$

Put Eqs. (1) and (4) side by side: brute force is $N \cdot D$, ANN is $(\log_2 N) \cdot D$. The D cancels in the ratio, so the asymptotic gap is

$$\frac{\text{ops}_{\text{brute}}}{\text{ops}_{\text{ANN}}} = \frac{N \cdot D}{\log_2 N \cdot D} = \frac{N}{\log_2 N}, \quad (5)$$

which itself grows without bound. Brute force does not just lose to ANN — it loses by a *widening* margin as the store grows.

Intuition

Replacing N with $\log_2 N$ is the whole game. Going from a thousand memories to a billion — a *million-fold* more data — raises $\log_2 N$ from about 10 to about 30: roughly $3\times$ more work, not $10^6\times$. ANN turns “proportional to everything stored” into “proportional to the number of *digits* in everything stored.”

5 Step 4 — the operations count, tabulated

Now the numbers, at $D = 768$ (a common embedding size) and $K = 5$. The brute-force column is $N \cdot D$ from Eq. (1); the ANN column is $\lceil \log_2 N \rceil \cdot D$ rounded from Eq. (4); the last column is their ratio, Eq. (5) (times D cancelling).

store size N	brute $N \cdot D$	ANN $\log_2 N \cdot D$	speedup
1,000	768,000	7,654	100×
1,000,000	768,000,000	15,307	50,172×
1,000,000,000	768,000,000,000	22,961	33,447,777×

The brute-force column climbs from three-quarters of a million ops to *three-quarters of a trillion* — a million-fold rise, exactly tracking N . The ANN column barely moves: from 7,654 to 22,961, a mere $3\times$ over the same span, because only $\log_2 N$ changed (from 9.97 to 29.90). Sample check, $N = 10^6$: $\log_2(10^6) = 19.93$, so ANN ops = $19.93 \times 768 = 15,307$, and brute = $10^6 \times 768 = 7.68 \times 10^8$, giving $7.68 \times 10^8 / 15,307 \approx 50,172\times$. ✓

Watch out

Per-turn latency is the recall cost multiplied by every turn of every session. A long-lived agent that recalls memory once per turn, on a store that grows unboundedly (the module’s *no-TTL* anti-pattern), pays $O(N \cdot D)$ that *climbs all year*. The agent gets slower the longer it has been useful — the worst possible coupling. ANN’s $O(\log N \cdot D)$ stays flat enough that a $1000\times$ bigger store costs only a handful more comparisons.

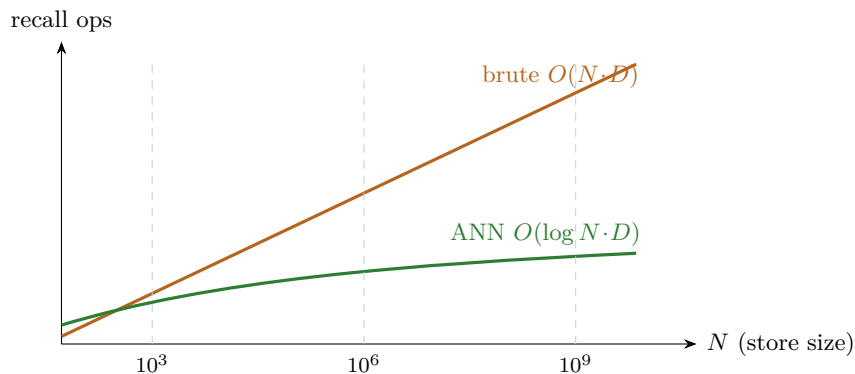
6 Step 5 — why brute force fails as memory grows

The module is explicit that long-term memory is “across threads, per user, lifetime indefinite” and warns against letting it “grow unboundedly forever.” Those two facts are exactly the conditions under which brute force fails:

- **N is unbounded and monotonic.** Every durable fact adds a vector; nothing is removed without a TTL. So N ratchets up over the agent’s lifetime.
- **Recall happens every turn.** Cost is paid per turn, so the per-turn latency of a brute-force store rises *linearly with everything the agent has ever stored*.

Multiply the two and a successful, long-running agent digs its own performance grave: the more it remembers, the slower each turn. ANN breaks the coupling by making recall depend on $\log N$ instead of N — the store can grow a millionfold and recall barely notices. (A TTL, the module’s other fix, attacks N directly; ANN and TTL are complementary.)

7 The scaling, drawn



The brute line rises in lock-step with the store; the ANN line is almost flat — the same $\frac{N}{\log_2 N}$ gap of Eq. (5), drawn. A store that grows a million-fold along the x -axis sends the brute curve through the roof while the ANN curve gains a sliver.

8 Retrieval cheat-sheet

One similarity (dot product over D)	$O(D)$
Brute-force similarity phase	$\text{ops} = N \cdot D = O(N \cdot D)$
Select top- K by full sort	$O(N \log N)$
Select top- K by size- K heap	$N \log_2 K = O(N \log K)$
ANN (HNSW) recall	$O(\log_2 N) \cdot D$
Brute / ANN gap	$\frac{N \cdot D}{\log_2 N \cdot D} = \frac{N}{\log_2 N}$
Effect of growing N	brute $\propto N$; ANN $\propto \log N$

9 An honest word about these numbers

The dimension $D = 768$ and the round $N \in \{10^3, 10^6, 10^9\}$ are illustrative anchors, and the ANN cost $O(\log N \cdot D)$ is an *asymptotic* model: a real HNSW index also has tunable knobs (`ef_search`, graph degree M) that set the constant, trades a little recall accuracy for speed (it is *approximate*), and pays a one-time build cost. We have also counted only the distance multiply-adds, not memory bandwidth or cache effects, which on real hardware can dominate. What is *not* illustrative is the shape: brute force is linear in N and ANN is logarithmic in N , so the gap $N/\log_2 N$ widens without bound as the store grows. Change D or the index parameters and the constants move; the linear-vs-logarithmic divide does not. That divide is why a long-lived agent's memory must be indexed, not scanned.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the operations-count table, the heap-vs-sort comparison, and the speedup column; change D or K to model your own embedding size and recall width.

```

1 # topk_retrieval.py -- reproduces every number in
2 # "Top-K Memory Retrieval: Scaling, by Hand".
3 import math
4
5 D = 768 # embedding dimension
6 K = 5 # top-K (store.asearch limit=5)
7
8 def brute_ops(N, D): # Eq. (1): N similarities, each O(D)
9     return N * D
10
11 def ann_ops(N, D): # Eq. (7): ~log2(N) candidate comparisons
12     return math.log2(N) * D
13
14 def sort_ops(N): # Eq. (2): full comparison sort
15     return N * math.log2(N)
16
17 def heap_ops(N, K): # Eq. (3): size-K heap selection
18     return N * math.log2(K)
19
20 print(f"D={D}, K={K}")
21 print(f"{'N':>14} | {'brute N*D':>16} | {'ANN log2(N)*D':>14} | {'speedup':>13}")
22 for N in (10**3, 10**6, 10**9):

```

```

23     bf, an = brute_ops(N, D), ann_ops(N, D)
24     print(f"{N:>14,} | {bf:>16,} | {an:>14,.0f} | {bf/an:>12,.0f}x")
25
26 print("\nselection: full sort vs size-K heap")
27 for N in (10**3, 10**6, 10**9):
28     print(f" N={N:>13,} sort={sort_ops(N):>16,.0f} heap={heap_ops(N,K):>14,.0f}")
29
30 # Expected output:
31 # D=768, K=5
32 # N | brute N*D | ANN log2(N)*D | speedup
33 # 1,000 | 768,000 | 7,654 | 100x
34 # 1,000,000 | 768,000,000 | 15,307 | 50,172x
35 # 1,000,000,000 | 768,000,000,000 | 22,961 | 33,447,777x
36 #
37 # selection: full sort vs size-K heap
38 # N= 1,000 sort= 9,966 heap= 2,322
39 # N= 1,000,000 sort= 19,931,569 heap= 2,321,928
40 # N=1,000,000,000 sort= 29,897,352,854 heap= 2,321,928,095

```

— end of document —